Reporte perceptrón multicapa.

Para esta actividad se hizo la comparación de dos implementaciones de un perceptrón multicapa, esto con el fin de clasificar las tres especies del conjunto Iris, una la cual fue desarrollada de manera manual con el uso de NumPy y otra haciendo uso de Keras, fueron realizadas pruebas con la arquitectura original (4x3x3) y una mucho más profunda (4x3x3x3x3), se mantuvieron las 500 épocas, la tasa de aprendizaje de 0.03, activación sigmoide y error MSE.

Resultados y comparaciones.

En NumPy se pudo obtener un error final de 0.059608 y una exactitud de 96.67%, mientras que la red profunda obtuvo un error 0.406424 y una exactitud de 70.67, demostrando que agregar capas no es necesario para poder mejorar el aprendizaje.

En Keras observamos que, el error final de la red original fue de 0.1886, mientras que el de la red profunda fue de 0.2222, comparando ambas implementaciones se puede determinar que hay presente un aumento del error al agregar 2 capas, empeorando el rendimiento, coincidiendo con el comportamiento del modelo de NumPy, en la cual la arquitectura profunda tampoco logró mejorar el aprendizaje.

Las curvas no necesariamente coinciden o son iguales, dadas las diferencias por las implementaciones, NumPy actualiza pesos por cada ejemplo, conservando el orden de los datos, por su otra parte, Keras trabaja lotes y mezcla los ejemplos durante el entrenamiento. También al utilizar diferentes inicializaciones de pesos puede producir distintas curvas de aprendizaje.

Una red más profunda puede terminar por aprender de peor forma, esto dado el uso de los sigmoides y MSE, esto debido que al acumular varias capas sigmoides los gradientes pueden volverse demasiado pequeños, lo cual dificulta la actualización de los pesos. Esto se determina como desvanecimiento del gradiente y logramos observarlo en el peor resultado dado por una red profunda de NumPy.

Conclusión.

Estos resultados demuestran que el agregar más capas no puede garantizar una mejor clasificación. Dado el conjunto Iris, una arquitectura sencilla logra ser más que suficiente y eficiente para ofrecer los mejores resultados ante una red profunda, las diferencias entre los modelos de NumPy y Keras son claro ejemplo de que la inicialización y el procedimiento de entrenamiento influyen en el aprendizaje de una red neuronal.